

Classifying Action Requirements in Short Text Messages Using Sentence Embeddings

Zuzanna Makowska

zuza.m.makowska@gmail.com

Abstract

This project explores the task of classifying short text messages based on whether they require user action. A custom dataset of 100 samples was created using an instruction-tuned large language model and manually refined to ensure diversity and consistency. Sentence embeddings were generated using a pre-trained transformer model (all-MiniLM-L6-v2), and several classifiers, including Logistic Regression, Linear SVM, and Random Forest, were evaluated on this representation.

The results show that Logistic Regression achieved the best performance, reaching an accuracy of 0.70 and a macro F1-score of 0.69. The final model was retrained on the full dataset and deployed as an interactive demo. The project demonstrates that even simple classifiers can effectively capture communicative intent when combined with semantic embeddings, while also highlighting the importance of dataset quality and careful use of AI tools during development.

1 Dataset

A custom dataset¹ including 100 samples was primarily generated using an instruction-tuned large language model. Each instance consists of a short sentence resembling real-world communication, such as system notifications, emails, or application prompts, and is assigned one of three labels: *action_required* (34 samples), *optional_action* (33 samples), or *informational* (33 samples).

To ensure diversity, the dataset includes variations in tone, phrasing, and context, reflecting different types of real-world communication. Following the generation process, the samples were manually reviewed and corrected where necessary.

¹<https://huggingface.co/datasets/McOwska/action-required-text-classification>

2 Models

2.1 Embeddings

Sentence embeddings were generated using the pre-trained SentenceTransformers model all-MiniLM-L6-v2, which maps input sentences to 384-dimensional dense vectors. The model was used as a fixed feature extractor without further fine-tuning. These embeddings capture semantic meaning, allowing the classifiers to operate on intent rather than surface-level lexical patterns.

2.2 Classification

Three classifiers were evaluated on top of the sentence embeddings: Logistic Regression, Linear Support Vector Machine (SVM), and Random Forest. The models were trained on the training split and evaluated on a held-out test set, with macro-averaged F1-score used as the primary selection metric.

The results indicate that Logistic Regression achieved the best overall performance, with an accuracy of 0.70 and a macro F1-score of 0.69. In comparison, both Linear SVM and Random Forest obtained lower performance, each reaching an accuracy of 0.64 and macro F1-scores of 0.63 and 0.64, respectively.

Based on these results, Logistic Regression was selected as the final model and retrained on the full dataset for deployment in the demo application². A working demo is available online³.

3 AI Assisted Development and Reflection

AI tools were used at different stages of the project, including dataset creation, implementation, and documentation.

²<https://huggingface.co/McOwska/action-requirement-classifier>

³<https://huggingface.co/spaces/McOwska/action-requirement-classifier-demo>

071 The dataset was generated using ChatGPT. The
072 main challenge was ensuring sufficient diversity
073 and difficulty of the samples. The initial version of
074 the dataset was too simple (samples from the same
075 category were too similar to each other), which re-
076 sulted in almost perfect accuracy for all models. To
077 address this, the dataset was iteratively improved
078 by introducing more varied and ambiguous exam-
079 ples.

080 For implementation, an AI coding assistant
081 (Codex) was used to help develop the training script.
082 A key aspect of working with the tool was pro-
083 viding clear and precise instructions, as well as
084 sufficient context about the project. In particular,
085 including example files helped guide the style and
086 structure of the generated code, making the outputs
087 more consistent with the intended implementation.
088 Additionally, when working with AI agents (espe-
089 cially when they are given a higher degree of au-
090 tonomy), version control systems proved especially
091 valuable. They made it easier to track changes, re-
092 vert undesired modifications, and maintain control
093 over the development process.

094 Finally, ChatGPT was also used to support writ-
095 ing the report and refining the README files. The
096 generated content was helpful, but (in the very hon-
097 est author's opinion) required some language ad-
098 justments to match the personal (and more human-
099 like) writing style.

100 Overall, AI tools were very helpful in speed-
101 ing up the development process, but they required
102 careful use, particularly when validating whether
103 the generated output aligns with the intended as-
104 sumptions and (maybe even above all) does not go
105 beyond the scope of the task.